CV Intern Task - Report

Egg Segmentation and Tracking

Tasks 1-4: Pretrained Segmentation, Classical Segmentation, Evaluation, and Video Tracking

Item	Details
Author	Sai Dhananjaya
Code repository	https://github.com/saidhanunjaya/nosh-egg-segmentation
Task 4 output video	https://drive.google.com/file/d/15EBvtN9CcjhM4N869Stfo_eskTJgfvig/view?usp=sharing

Contents

- **Section 1** - Task 1: Pretrained-model egg segmentation (OWL-ViT + SAM)
- **Section 2** - Task 2: Classical image-processing segmentation and experiment log
- **Section 3** - Task 3: Quantitative and qualitative evaluation
- **Section 4** - Task 4: Egg tracking in video
- **Section 5** - Setup and reproduction
- **Section 6** - AI tools disclosure

Section 1: Task 1 - Pretrained-Model Egg Segmentation

1.1 Objective

The goal of Task 1 was to segment individual eggs in a dataset of real-world photos using pretrained, off-the-shelf models — no custom training or fine-tuning. The output of this task (instance masks) serves two purposes downstream: it is treated as **ground truth** for evaluating the classical algorithm built in Task 2 (see Task 3), and it establishes the dev/eval split reused throughout the project.

1.2 Pipeline

A two-stage, zero-shot detect-then-segment pipeline was used:

1. **Detection — OWL-ViT (open-vocabulary object detector).** Each image is passed through OWL-ViT with the text prompt `"egg"`. OWL-ViT is a text-prompted detector, able to localize objects matching a natural-language description without being trained specifically on eggs. This produces bounding boxes around candidate egg regions, with confidence scores.
2. **Segmentation — SAM (Segment Anything Model), box-prompted.** Each OWL-ViT bounding box is passed as a prompt into SAM, which produces a precise pixel-level instance mask for the object inside that box.

This combination allows fully automatic, label-free instance segmentation: OWL-ViT answers "where are the eggs," and SAM answers "what exactly are their pixel boundaries."

A lightweight **fallback mechanism** supplements primary OWL-ViT detections: when the primary detector's output is missing or low-confidence for an image, a fallback box-proposal step (`source: fallback-1` in `review_list.txt`) supplies a substitute box, and the image is automatically flagged for manual review so this substitution is never silent.

1.3 Dataset and Train/Eval Split

The dataset (Roboflow export, egg images across varied backgrounds — hands, trays, marble, cloth, keyboards, leaves, etc.) contains 110 total images after de-duplication of Roboflow's augmented copies.

To prevent data leakage (Roboflow's augmentation pipeline can produce multiple near-duplicate copies of the same original photograph), images were grouped by **original source photo** before splitting, and a 4:1 grouped split was applied:

- Total images: **110**
- Dev set: **88** images
- Eval set: **22** images

This exact split (recorded in `task1_outputs_final/split.csv`) is reused unchanged in Task 2, so the classical algorithm is developed and evaluated on the same images as the pretrained pipeline, making the Task 3 comparison fair.

1.4 Outputs

For each image, the pipeline produces:

- `masks/` — binary instance mask(s), one per detected egg
- `labels/` — class/instance label files
- `instances/` — per-instance crops/metadata
- `overlays/` — the original image with mask(s) drawn on top, for visual QA
- `diagnostics/` + `review_list.txt` — images flagged for manual review (fallback detections or manually-corrected boxes, `source: fallback-1` / `source: manual`)
- `results.csv` — per-image summary

1.5 Example Results

Figure 1 (clean result)

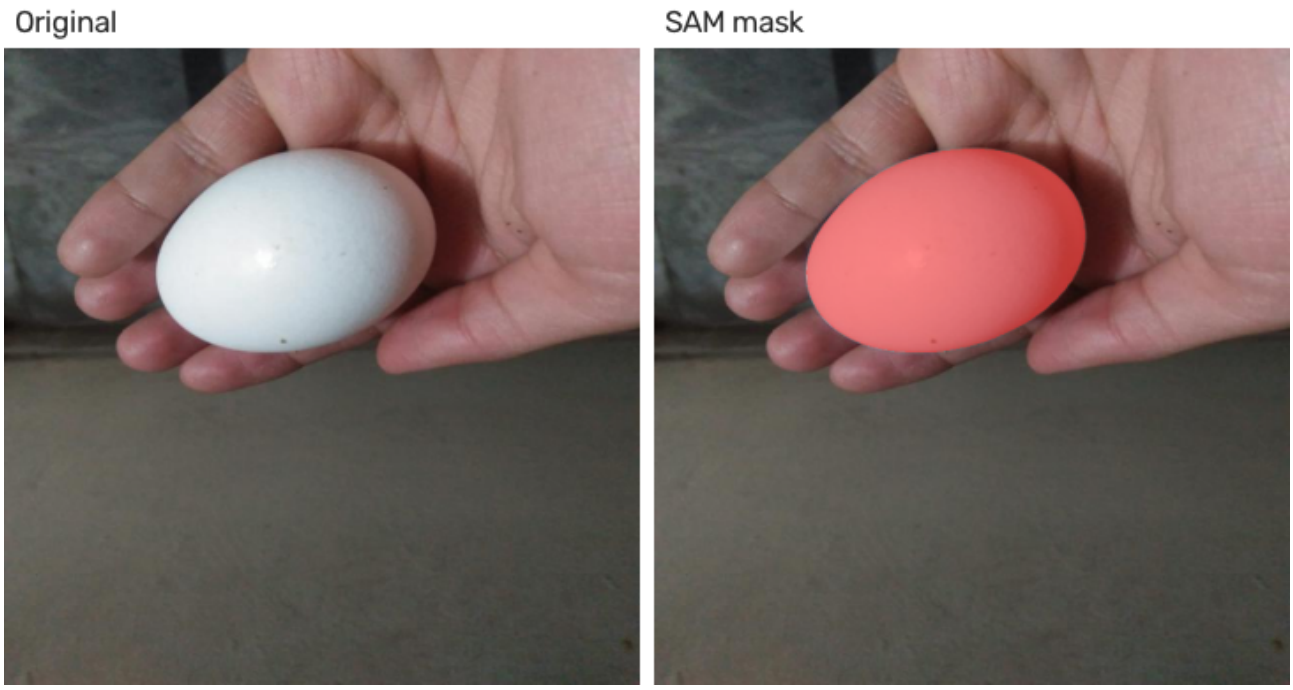


Figure 1: A clean, tight segmentation - OWL-ViT correctly localizes the egg held in a hand, and SAM produces an accurate mask over it.

Figure 2 (occlusion artifact)



Figure 2: The egg is partially occluded by a crossing plant stem. SAM's mask boundary is disrupted at the occlusion point, illustrating a minor limitation under occlusion.

Figure 3 (fallback detection, flagged)

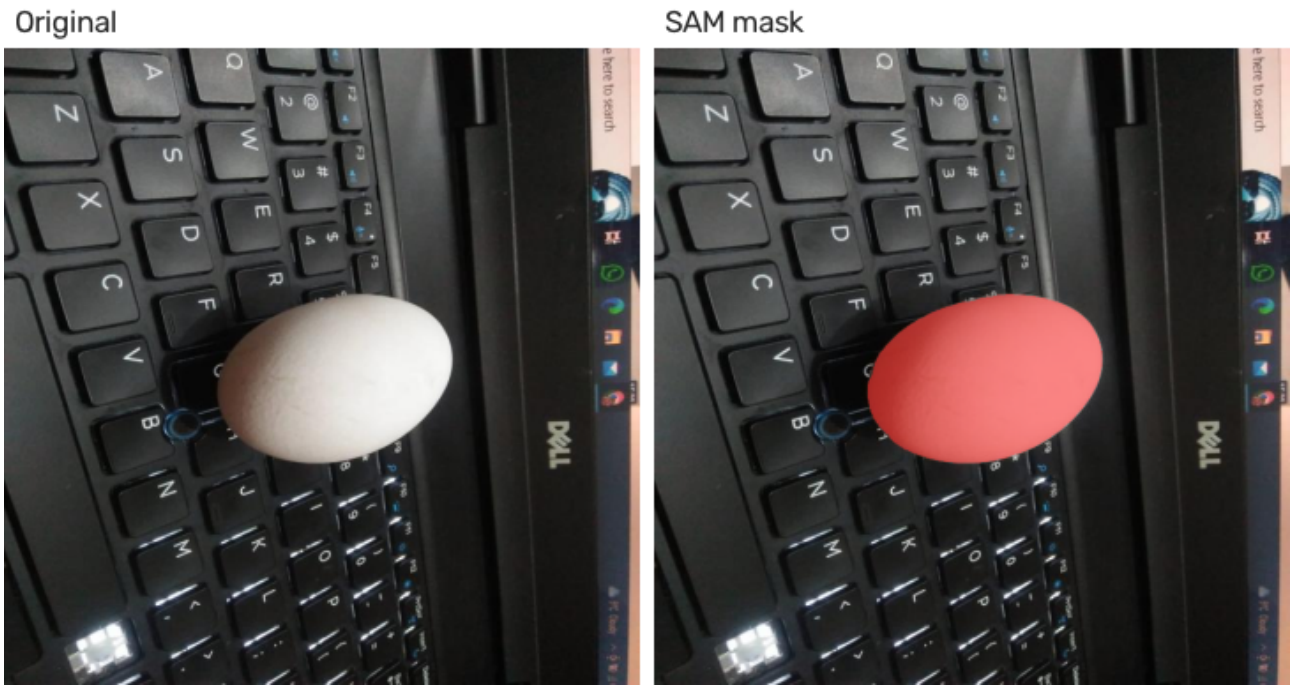


Figure 3: Despite a visually clean result, this image's box came from the fallback detector rather than primary OWL-ViT output, and was automatically flagged in review_list.txt.

Figure 4 (unflagged failure)

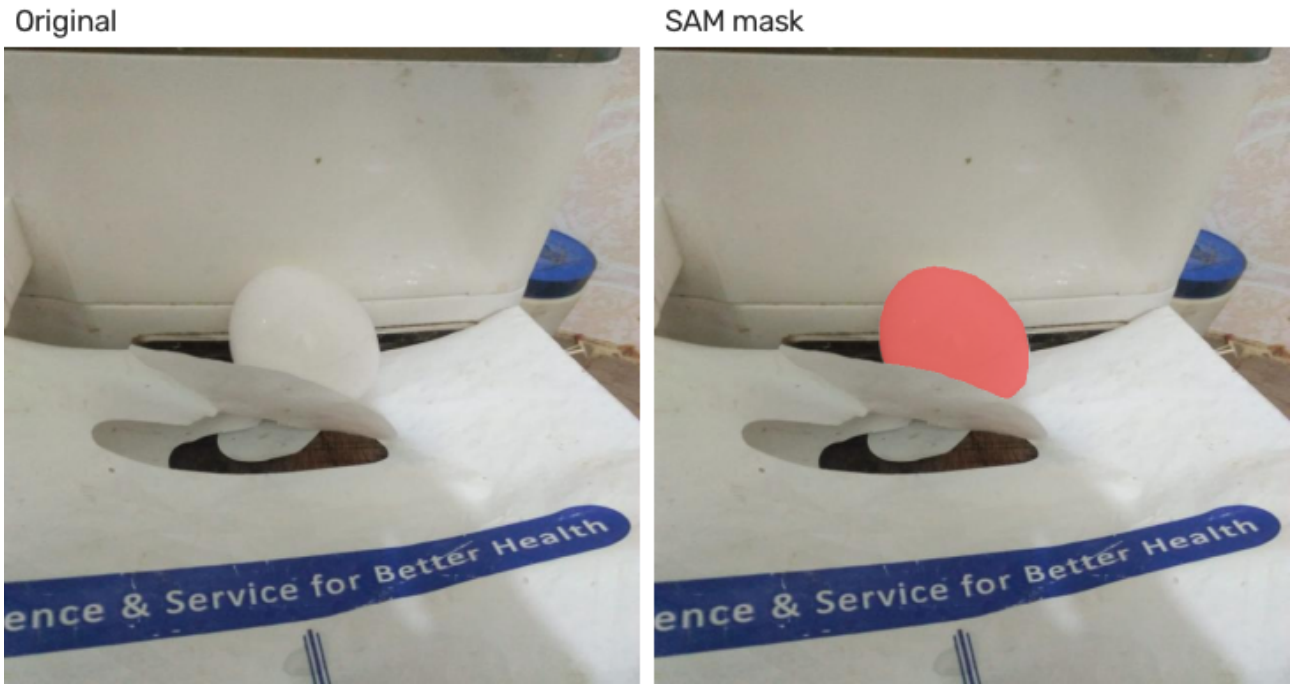


Figure 4: A genuine failure case - the segmented region sits on a fold of tissue paper, not an egg. Unlike Figure 3, this image was NOT flagged for review, a silent failure.

1.6 Notes and Limitations

- As a zero-shot pipeline, quality depends on OWL-ViT's detection recall/precision for the word "egg." A fallback mechanism catches some low-confidence cases and flags them for review (Figure 3), but is not a complete safety net: Figure 4 shows a confidently-wrong mask that passed through unflagged, meaning the review process does not catch 100% of real failures. Any downstream use of Task 1 outputs as "ground truth" (as in Task 3) should account for this.

- Multi-instance images (multiple eggs in frame) are handled naturally since OWL-ViT can return multiple boxes per image, each independently segmented by SAM — this differs materially from Task 2's classical pipeline, which has no native multi-instance handling (discussed in Task 3).
- No console logs/timing/threshold values were retained from the original run beyond the split statistics above; this section is written from the saved outputs (split.csv, review_list.txt, overlays) rather than run logs.

Section 2: Task 2 - Classical Image-Processing Egg Segmentation

2.1 Objective

The goal of Task 2 was to build a **classical (non-deep-learning) image-processing algorithm** that segments eggs as closely as possible to the quality of Task 1's pretrained OWL-ViT + SAM pipeline, using only traditional computer vision techniques (thresholding, edge detection, color-space operations, morphological operations, etc.) — no neural networks, no learned weights.

To keep the comparison in Task 3 fair, Task 2 reuses **the exact same dev/eval split** established in Task 1 (`task1_outputs_final/split.csv`: 110 total images, dev=88, eval=22). All development and trial-and-error below was performed on the **dev set only**; the eval set was not used to tune any parameter, and the final locked-in algorithm was run once over the full 110-image dataset for reporting.

It was expected going in — and is confirmed by Task 3's quantitative analysis — that a classical pipeline would **not** match SAM's pixel-level accuracy. The goal was to get as close as a classical approach reasonably can, build genuine intuition about why it falls short, and document that gap rigorously. That gap is the subject of Task 3.

2.2 Experiment Log

Nine trials were run in sequence on the dev set, each building on diagnosis of the previous trial's failures. The full log — including abandoned approaches — is reproduced below.

Trial	Change	Why Tried	Dev-Set Result	Verdict
1	Global Otsu thresholding on grayscale image	Simplest possible baseline — test if a single global threshold separates egg from background	Worked only on a handful of high-contrast, plain-background images; failed when background brightness overlapped egg brightness	Discarded — too simplistic, no spatial reasoning
2	Otsu + morphological clean-up + "pick most egg-shaped blob" scorer	Add shape-based selection to reject non-egg-shaped blobs from noisy segmentation	Marginal improvement; threshold itself remained the bottleneck	Discarded — downstream filtering can't fix an unreliable upstream threshold
3	Canny edge detection to contour extraction to ellipse-fit scoring (RETR_EXTERNAL)	Eggs are elliptical; edge+ellipse fitting should be more lighting-robust than intensity thresholding	Found edges often, but shadows/texture created spurious contours; false "egg-shaped" picks on clutter	Discarded as-is, iterated further (3b-3e)
3b	Switched to RETR_CCOMP to also search holes/internal contours	Hypothesis: true egg boundary sometimes appears as an internal contour	Recovered a few missed cases, introduced new false positives from internal texture (cloth folds, marble veining)	Discarded — net neutral, instability increased

Trial	Change	Why Tried	Dev-Set Result	Verdict
3c	Added relative-size filter on contours	Prune obviously wrong-scale false candidates from 3b	Removed some false positives, but filtered out legitimate eggs at unusual scales	Discarded — couldn't separate true-but-odd-scale from false-but-plausible-scale
3d	Added absolute HSV brightness filter	Exploit that eggs are often lighter than many backgrounds	Worked on tuning images only; broke on darker eggs / bright backgrounds (skin, pale tile, keyboard highlights)	Discarded — absolute thresholds don't generalize across lighting diversity
3e	Switched to relative brightness/uniformity filter	Make the brightness heuristic lighting-invariant	Better than 3d but still failed on ash/rock textures, hand creases, keyboard gaps, patterned cloth	Discarded — confirmed local shape+color+brightness heuristics cannot generalize across this dataset
4	Combined: accept if EITHER Otsu blob OR Canny/ellipse contour scored well	Hypothesis: the two methods fail on different images; combining covers more cases	Some images improved, but combining two brittle methods mostly combined their failure modes (more false positives)	Discarded
5	Lighter variant of Trial 4: tuned closing + combined scoring	Last attempt before abandoning the family — rule out that a lighter touch fixes brittleness	Continued failing on same categories as 3e; each fix for one image broke a previously-working one	**Entire contour/shape-scoring family abandoned.** Root cause: local heuristics are fundamentally brittle and cannot generalize across this dataset's background diversity
6	GrabCut seeded with plain centered rectangle + YCrCb skin-removal post-step	GrabCut models global fg/bg color distributions — fundamentally different from local shape heuristics	1/8 clean, 1/8 near-success; 2/8 partial failures from skin-removal over-firing on pale egg highlights; 3-4/8 severe background bleed; 1/8 total fg/bg inversion on patterned cloth	Kept concept; diagnosed two bugs: (A) skin mask too broad, catches pale egg highlights; (B) plain rectangle seed gives no guaranteed-fg anchor, allows inversion
7	Trimap seeding with hard GC_FGD center core + saturation-gated skin removal	Directly address both Trial 6 bugs	Net regression — 0/8 improved, several much worse; one hand image produced a totally unrelated diagonal wedge mask	Discarded — hard FGD core poisons the color model when background pixels fall inside the box (nearly always); largest-component pick can select an unrelated object if skin removal fragments the true foreground
8	Soft trimap (hard GC_BGD border only, no hard foreground) + one-time retry if fg >55% of frame + saturation-gated skin removal + pick component closest to image center (not largest)	Fix Trial 7's two bugs directly	Substantial improvement — most images now produce a plausible single egg-centered region; bleed reduced but not eliminated; residual thin "neck" connections remained	**Kept** — best result so far, neck-shaped bleed artifacts remained

Trial	Change	Why Tried	Dev-Set Result	Verdict
9	Added morphological opening (kernel approx 2% of image diagonal) to sever thin necks, then re-apply centroid-pick and hole-filling	Opening removes thin connecting structures while preserving the wider main blob	Fixed several neck-connected cases cleanly; confirmed opening did NOT sever wide/stubby necks on marble, ash/rock, steel-pan backgrounds; widening kernel further risked eroding smaller eggs	**Locked in as final.** Accepted as a real technique ceiling: opening helps thin-neck bleed but not wide-neck bleed

Trial budget note: after Trial 7's regression, a hard budget of 2-3 further trials was agreed before locking in whatever variant performed best. Trial 9 was the final trial under that budget.

2.3 Final Algorithm (Locked In)

The final pipeline, implemented in `task2_segment_eggs.py`, is:

1. **Soft-trimap-seeded GrabCut** — only the border region outside a detection box is marked hard background (GC_BGD); everything inside is left as "probable" fg/bg for GrabCut to resolve iteratively. No hard foreground core is used.
2. **One-time adaptive retry** — if the resulting foreground covers more than 55% of the frame, the box is tightened and GrabCut is re-run once.
3. **Saturation-gated YCrCb skin removal** — skin-colored pixels are removed from the foreground only above a saturation threshold, protecting pale egg highlights.
4. **Morphological opening** (kernel approx 2% of image diagonal) to break thin background-bleed "necks."
5. **Centroid-based component selection** — the connected component whose centroid is closest to the image center is kept, not the largest by area.
6. **Hole-filling** to produce a clean, solid final mask.

Output structure mirrors Task 1 (`task2_outputs_final/masks/`, `labels/`, `instances/`, `overlays/`, `results.csv`, `review_list.txt`), with `results.csv` additionally logging `area`, `frame_frac`, `retried`, `flagged`, and `flag_reason` per image for transparency.

2.4 Full-Dataset Results

The final script was run once over the complete 110-image dataset (both dev and eval), with no crashes:

- **110/110 images processed successfully**
- **39/110 (35.5%) flagged for review**, predominantly due to suspected background bleed
- **2/110 complete failures** (area = 0): a three-eggs-on-marble scene, and a hand+egg scene on low-contrast tile floor
- An additional confirmed qualitative failure: a foil snack packet fused with the egg through a thick color-similarity "neck," surviving the morphological opening step due to its width

2.5 Known Limitations

- **No native multi-instance handling.** The pipeline selects exactly one connected component per image by design. Scenes with multiple eggs will have at most one correctly captured — a structural limitation, and a major root cause behind several of Task 3's failure categories (see Section 3.3.1).
- **Global color-model bleed.** GrabCut's foreground/background color model is global per image; when the egg's color is statistically similar to a large part of the background, the foreground region can bleed into or fuse with it. The morphological-opening fix only resolves thin-neck cases; wide necks are a confirmed, accepted failure mode.
- **No semantic understanding of "egg."** Unlike Task 1's OWL-ViT detector, the classical pipeline has no concept of what an egg looks like — it can confidently segment a non-egg object if that object's color statistics happen to separate well from its local surroundings.

- Parameter choices were tuned on the dev set by visual inspection, not formal grid search or cross-validation — consistent with the trial-and-error approach documented above.

Section 3: Task 3 - Quantitative and Qualitative Evaluation

3.1 Methodology

Task 2's classical segmentation masks were evaluated against Task 1's SAM-based masks, which were treated as the reference masks. The two masks for each image were paired by filename stem and compared pixel by pixel using Intersection-over-Union (IoU), Dice coefficient, precision, and recall.

For predicted mask P and reference mask G:

- **IoU** = |P intersection G| / |P union G|
- **Dice** = 2 |P intersection G| / (|P| + |G|)
- **Precision** = TP / (TP + FP)
- **Recall** = TP / (TP + FN)

The reported summary statistics are calculated across the 110 per-image results. Task 1 masks are treated as operational reference masks, not manually verified ground truth. As noted in Section 1, Task 1 contains at least one unflagged failure; therefore, these metrics measure agreement with Task 1 rather than absolute accuracy against human-annotated masks.

Visualization lookup bug, found and fixed. The original evaluation script's image-lookup function assumed that source images were stored in a flat folder. The Roboflow export instead nests images one level deeper. This caused the visualization step to fail to locate source images, leaving the `worst_cases` folder empty even though the script reported success. The bug affected visualization only: the metrics were computed from mask files paired by filename, and there were zero missing or unmatched masks. The image lookup was fixed by using a recursive search, after which the generated worst-case comparisons were manually inspected.

Verifying zero-IoU failures. Some images had both precision and recall equal to zero. This can occur when the prediction is a non-empty mask that does not overlap the reference mask at all. Manual inspection of the zero-IoU examples confirmed that these were genuine wrong-region predictions, not a pairing error: the Task 1 mask was on the egg, while the Task 2 prediction was on unrelated regions such as a printed graphic, tray rail, fabric seam, floor shadow, cable, or label.

The results below summarize all 110 images. Task 2's parameters were tuned on the 88-image dev set, with the 22-image eval set held out during development. Because the table reports the pooled full-dataset results rather than a separate eval-only score, these figures should be read as a full-dataset comparison, not as an independent holdout estimate.

3.2 Quantitative Results

Metric	Mean	Median
IoU	0.3436	0.2685
Dice	0.4422	0.4234
Precision	0.4312	0.2873
Recall	0.7217	0.9003

Per-image IoU ranged from 0.0000 (minimum) to 0.9943 (maximum).

IoU range	Images	Percentage
IoU >= 0.75 - good	19 / 110	17.3%
0.50 <= IoU < 0.75 - moderate	7 / 110	6.4%
IoU < 0.50 - poor or failed	84 / 110	76.4%

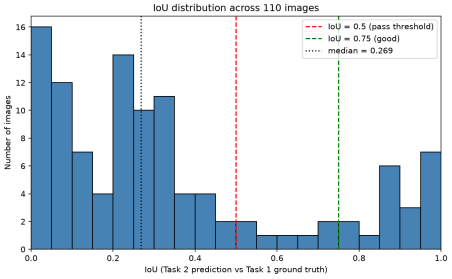


Figure 5 - IoU distribution. *A large concentration of low-IoU results and a smaller group of strong results: performance varies substantially between images.*

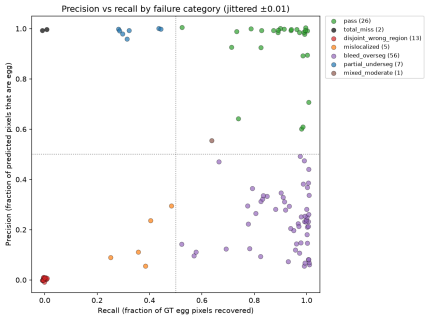


Figure 6 - Precision and recall per image. *Recall is generally higher than precision, so over-segmentation is a more common error than missing egg pixels.*

Mean recall (0.7217) is substantially higher than mean precision (0.4312); the median values show the same pattern. This suggests that the classical pipeline often captures some or most of the reference egg region, but includes too much background in its prediction. This is consistent with the background-bleed behaviour observed during Task 2 development.

3.3 Failure-Mode Taxonomy

Each image was assigned to one of seven mutually exclusive categories using a fixed set of rules applied in priority order.

Category	Description	Images	Percentage	Mean IoU
`pass`	Strong agreement with the reference mask	26	23.6%	0.833
`total_miss`	The egg region was effectively missed	2	1.8%	0.000
`disjoint_wrong_region`	The prediction landed on a separate, unrelated region	13	11.8%	0.000
`mislocalized`	The predicted region was misplaced, with limited overlap	5	4.5%	0.119
`bleed_overseg`	The egg was partly captured, but the mask spilled into background	56	50.9%	0.228
`partial_underseg`	Only part of the reference region or one of several eggs was captured	7	6.4%	0.339
`mixed_moderate`	A combination of moderate segmentation errors	1	0.9%	0.415

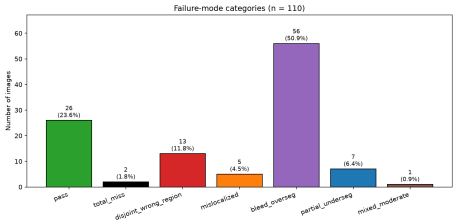


Figure 7 - Images per category. *`bleed_overseg` is the largest category, accounting for 56 of 110 images.*

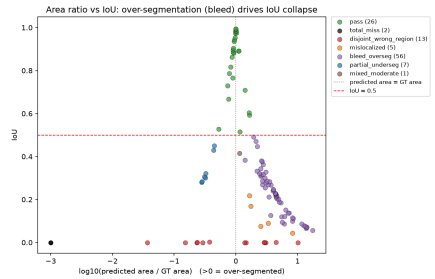


Figure 8 - Area ratio versus IoU. *Many low-IoU `bleed_overseg` predictions have a much larger area than the reference mask.*

`bleed_overseg` is the dominant failure mode, affecting 50.9% of the images. Its median predicted-area-to-reference-area ratio is 4.21, meaning that the predicted masks in this category are typically several times larger than the reference egg region. This provides quantitative evidence of the background bleed observed in Task 2.

3.3.1 Separating the Main Contributing Factors

To examine the effect of Task 2's single-component design, each Task 1 reference mask was checked for multiple instances. An image was counted as multi-instance when the reference mask contained at least two connected components, each occupying at least 3% of the mask area.

Category	Images in category	Multi-instance images	Multi-instance rate
`pass`	26	3	11.5%
`total_miss`	2	1	50.0%
`disjoint_wrong_region`	13	1	7.7%
`mislocalized`	5	3	60.0%
`bleed_overseg`	56	3	5.4%
`partial_underseg`	7	7	100.0%
`mixed_moderate`	1	0	0.0%

The dataset's overall multi-instance rate is **16.4% (18/110 images)**. The results point to two main contributing factors. These factors can co-occur in an image; the table above assigns each image to only one failure category.

- **Factor A - single-object color-model failure.** The `bleed_overseg` and `disjoint_wrong_region` categories contain 69 images (62.7% of the dataset). Their multi-instance rates (5.4% and 7.7% respectively) are below the dataset baseline. This suggests that the dominant bleed and wrong-region failures are generally not explained by multiple eggs; they are more consistent with GrabCut's color model spilling into similar-looking background or selecting an unrelated region.
- **Factor B - no multi-instance handling.** All 7 `partial_underseg` cases are multi-instance images. Multi-instance scenes also occur in 3 of the 5 `mislocalized` cases and 1 of the 2 `total_miss` cases. That is 11 images (10.0% of the dataset) in these failure categories where a multi-instance scene coincides with a failure associated with missing or mislocalizing eggs. Task 2's design selects one connected component per image, so it cannot reliably represent multiple separate eggs.

3.4 Qualitative Analysis

The following examples illustrate the main failure categories and contrast them with stronger results. Figures 9-11 show background bleed and wrong-region predictions (Factor A), Figures 12-13 show multi-instance failures (Factor B), and Figures 14-15 show stronger results. In each image, yellow marks agreement between the prediction and the reference, green marks reference pixels that were missed, and red marks false-positive prediction pixels.



Figure 9. `not_damaged_43` (IoU = 0.269, precision = 0.291, recall = 0.782). The egg is captured, but the predicted mask also spreads across the laptop body through a dark, connected region.



Figure 10. `not_damaged_135` (IoU = 0.057, precision = 0.057, recall = 1.000). The reference egg is contained in the prediction, but a large false-positive region around the bead-filled bucket overwhelms the mask.

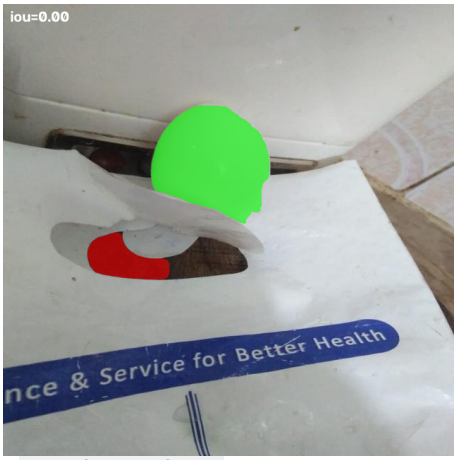


Figure 11. `not_damaged_123` (IoU = 0.000). The prediction falls on a plastic rail of the egg tray, while the reference egg region is untouched - a complete wrong-region capture.



Figure 12. `not_damaged_114` (IoU = 0.170, precision = 0.228, recall = 0.400). In this multi-egg marble scene, one egg is partly captured while the other eggs are missed.



Figure 13. `not_damaged_133` *(IoU = 0.322, precision = 0.995, recall = 0.322). The selected egg is segmented cleanly, but the remaining eggs are missed, resulting in low recall.*



Figure 14. `not_damaged_34` *(IoU = 0.994). This is a near-pixel-perfect match with the Task 1 reference mask.*

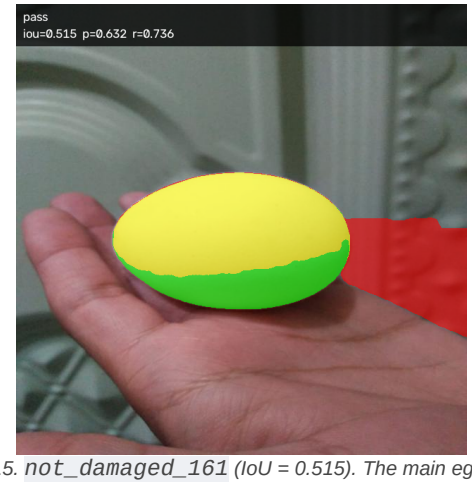


Figure 15. `not_damaged_161` *(IoU = 0.515). The main egg region is located, but part of the lower boundary is missed, likely because the shadowed rim is difficult to distinguish using the colour model.*

3.5 Discussion and Suggested Improvements

The quantitative and qualitative results identify two main improvement areas:

- **Reduce bleed and wrong-region captures.** GrabCut relies on foreground and background colour statistics and does not understand object boundaries or the concept of an egg. An edge-aware refinement step, boundary constraint, or saliency/objectness prior could help prevent the mask from spreading into similarly coloured backgrounds or settling on an unrelated region.
- **Support multiple eggs.** The current algorithm retains one connected component per image. Retaining multiple sufficiently large candidate components, followed by appropriate filtering, could allow the algorithm to represent several eggs in the same frame and address the `partial_underseg` cases.

These improvements were not implemented within the Task 2 trial budget. The results show that the final classical pipeline can produce strong masks under favourable conditions, but its global colour model and single-component selection impose clear limits on varied backgrounds and multi-egg scenes.

Section 4: Task 4 - Egg Tracking in Video

4.1 Objective

Run an egg segmentation and tracking pipeline on the provided YouTube video and keep a separate identity for each egg. I used the clip from 0:09 to 0:40 (31 s, 1280x720, 24 fps, 744 frames). The clip contains two eggs: one on the table and one held in a hand.

4.2 Why the Task 2 algorithm was not reused

Section 3 shows that the Task 2 pipeline over-segments often (mean IoU 0.3436 against the Task 1 masks, bleed/over-segmentation in 50.9% of images). For the video I wrote a simpler detector that outputs boxes only, since tracking needs object positions rather than exact masks. Its thresholds were tuned on this one clip.

4.3 Detector

File: `task4_step2_detector.py`. Steps per frame:

- Grayscale, then a fixed threshold of 150.
- Only the region below the table edge (`TABLE_TOP_Y = 255`) is searched.
- Morphological open then close, 5x5 elliptical kernel, 2 iterations each.
- Contour filters: area 3000-50000 px, minimum side 60 px, circularity >= 0.60, aspect ratio <= 1.8, mean HSV saturation <= 55 (eggs are white and low in saturation).

The detector finds the egg on the table reliably. It rarely finds the held egg. Lowering the threshold to 100 merges the egg with the hand, wall and shirt. A threshold of 150 fragments it, and a saturation mask (S < 12 and V > 110) isolates only the table egg. The causes are finger occlusion, motion blur and a bright background. This is the main reason a short-term tracker was added (Section 4.5).

4.4 Tracker

File: `sort_tracker.py`, a custom SORT implementation. Each track has a Kalman filter (predict, then update). Detections are matched to predicted boxes by IoU using the Hungarian algorithm. Parameters: `max_age=15, min_hits=2, iou_threshold=0.25, lost_buffer_age=120`.

I made four changes to the base tracker:

- Keep a `last_bbox` for each track.
- Set velocity to zero when a track was not updated, so predictions do not drift.
- Store dead tracks with their `last_bbox` instead of a drifted prediction.
- Output only tracks updated in the current frame (`time_since_update == 0`) with enough hits, so coasting tracks never produce ghost boxes.

Dead tracks are kept for `lost_buffer_age` frames and are revived when a new detection appears near the last known position, reusing the old ID. So the tracker reports 7 tracks born, but only 2 distinct IDs are ever drawn.

4.5 Short-term tracking with CSRT

File: `task4_step4_csrt.py` (requires `opencv-contrib-python`). When the detector loses an egg that already has an ID, a CSRT tracker started from its last detected box follows it and draws a thin box labelled "(est.)". Guards that stop CSRT from drawing wrong boxes:

- Maximum coasting of 120 frames (5 s).
- The box area must stay within 0.4 to 2.5 times the last detected area.
- The estimated box is dropped if it overlaps another egg's box with IoU > 0.30.
- The box centre must stay inside the frame.

- After CSRT gives up, it only restarts on the next real detection.

4.6 Development log

#	Change or trial	Observation	Decision
1	Threshold 150 plus shape and saturation filters	Reliable on the table egg, misses the held egg	Kept
2	Threshold lowered to 100 to catch the held egg	Egg merges with hand, wall and shirt	Discarded
3	Saturation mask (S < 12, V > 110)	Isolates only the table egg	Discarded
4	Plain SORT output	Ghost boxes: coasting tracks were drawn	Fixed: output only tracks updated this frame
5	Zero velocity when not updated, store `last_bbox`	Predicted boxes no longer drift away	Kept
6	`lost_buffer_age = 120` with revival	Same ID returns when the egg reappears (7 born, 2 IDs)	Kept
7	First CSRT script	Bug: drawn boxes leaked into the tracker input frame	Fixed with `clean = frame.copy()`
8	CSRT guards (area ratio, overlap, centre, max coast)	No stray "(est.)" boxes in the full video	Kept

4.7 Results

There are no ground-truth annotations for the video, so I report box coverage and visual checks. The maximum possible is 744 frames x 2 eggs = 1488 egg-frames.

Quantity	Egg-frames	Share of 1488
Boxes from detector + tracker	1231	82.7%
Boxes filled by CSRT	223	15.0%
Total with a box	1454	97.7%
No box	34	2.3%

I watched the full output video. I saw no label swaps between the two eggs, and no "(est.)" box remained on the hand, shirt or background after an egg left, although during holding the estimated box can be offset from the egg (Figure 4.2). The roles of Egg 0 and Egg 1 (table or held) swap over time because the eggs are exchanged in the video. This is not a label swap: the IDs follow the earlier ID segments.

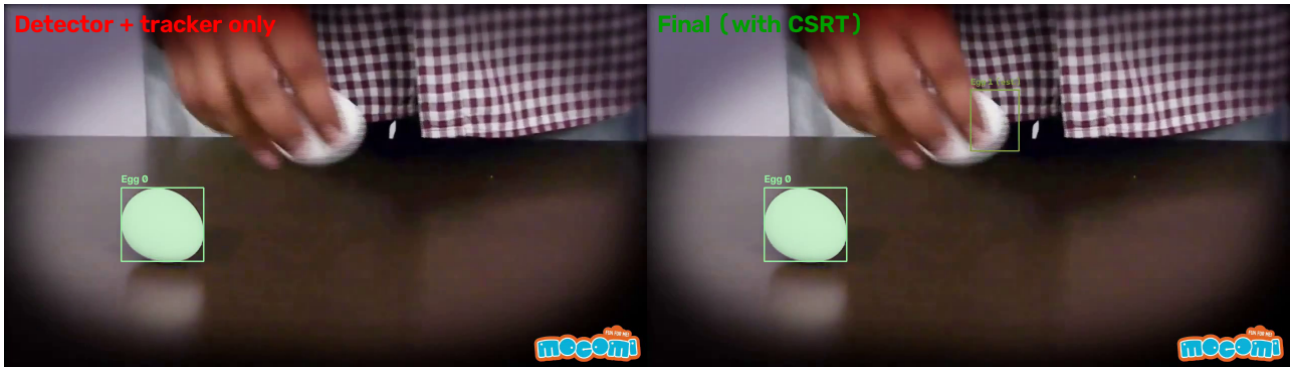


Figure 4.1 - t = 11.5 s: held egg, detector and tracker only (left) and final output with CSRT (right). The estimated box covers the egg but is slightly shifted to its right.

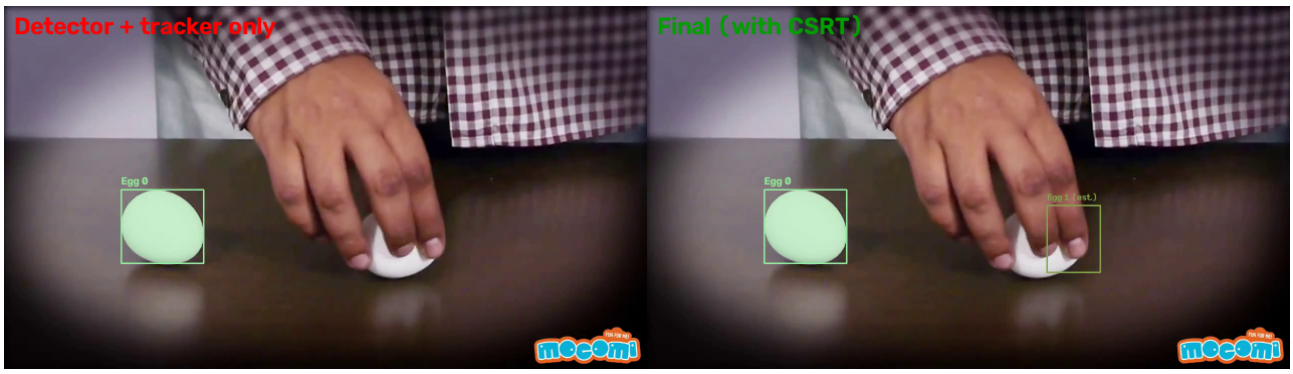


Figure 4.2 - t = 12.5 s: the CSRT box is offset toward the fingers and covers only part of the egg.

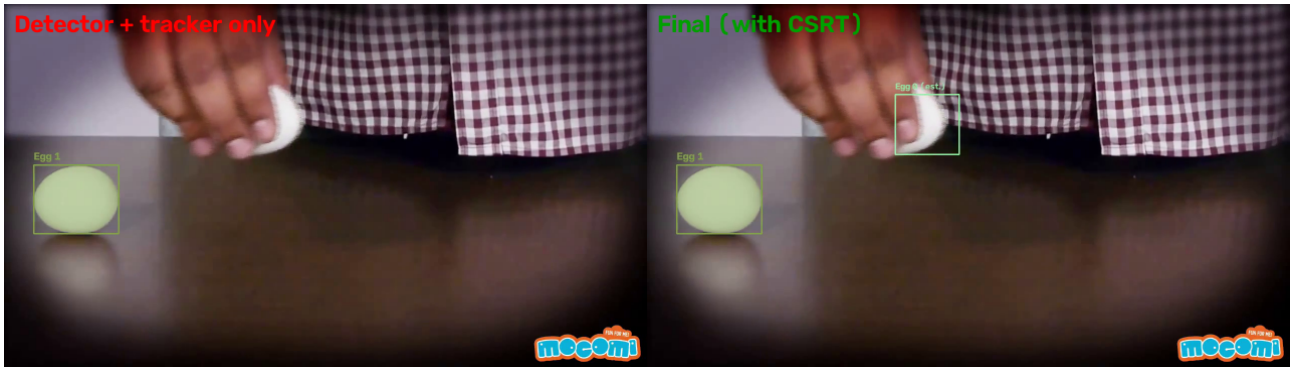


Figure 4.3 - t = 21.5 s: the other egg (Egg 0) is held and the CSRT box fits it well. Egg 1 is now the one on the table.

4.8 Output video

The final video (detector + tracker + CSRT) is available at: https://drive.google.com/file/d/15EBvtN9CcjhM4N869Stfo_eskTJgfvig/view?usp=sharing

Solid boxes come from the detector. Thin boxes labelled "(est.)" come from CSRT, meaning the egg was not detected in that frame and the position is an estimate.

4.9 Limitations

- CSRT boxes are approximate. They keep the right ID on the held egg, but they are not tight and can be offset toward the fingers (visible at 12.5 s in Figure 4.2). I did not measure box accuracy against annotations.
- The detector almost never finds the held egg, so the held egg is mostly tracked by CSRT.
- Thresholds and filters were tuned on this one clip and may not transfer to other videos.
- The setup assumes at most two eggs. The detector never produced three or more in this clip.
- There is no quantitative tracking metric (such as MOTA), because I have no tracking annotations.

4.10 Possible improvements

- Use the Task 1 model (OWL-ViT + SAM) as the detector on video frames, so the held egg is detected directly.
- Annotate a few frames by hand and measure the IoU of the CSRT boxes.
- Add appearance features to the association step for crowded scenes.

Section 5: Setup and Reproduction

Tested on Windows with Python and a virtual environment.

```
git clone https://github.com/saidhanunjaya/nosh-egg-segmentation.git
cd nosh-egg-segmentation
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Notes: Task 4 needs `opencv-contrib-python` for CSRT. Videos (`*.mp4`), the dataset folders and model weights (`*.pth`, `*.pt`) are not stored in the repository. Download the dataset and the SAM checkpoint separately. The Task 4 clip is the 0:09-0:40 part of the video, saved as `task4_trimmed.mp4`.

Task	Command	Output
Task 1 segmentation (OWL-ViT + SAM)	`python task1_segment_eggs.py`	`task1_outputs_final/`
Task 1 report figures (1-4)	`python task1_make_figures.py`	`task1_figures/`
Task 2 dev/eval split	`python task2_step0_split.py`	split of the 110 images (dev 88 / eval 22)
Task 2 trials 1-9 (experiment log)	`task2_step1_otsu.py` ... `task2_step9_openbreak.py`	dev-set trial outputs
Task 2 final algorithm	`python task2_segment_eggs.py`	`task2_outputs_final/`
Task 3 metrics	`python task3_evaluate.py`	`task3_outputs/`
Task 3 failure categories	`python task3_failure_modes.py`	category table
Task 3 multi-instance check	`python task3_multiinstance_check.py`	Section 3.3.1 table
Task 3 plots	`python task3_plots.py`	`task3_outputs/plots/`
Task 3 qualitative examples	`python task3_qualitative.py`	`task3_outputs/qualitative/`
Task 4 detection + tracking	`python task4_step3_track.py`	`task4_output.mp4`
Task 4 with CSRT	`python task4_step4_csrt.py`	`task4_output_csrt.mp4`
Task 4 comparison figures	`python task4_make_figures.py`	`task4_figures/`
Build this report	`python build_pdf.py`	`report.pdf`

Section 6: AI Tools Disclosure

I used Claude (by Anthropic) as an assistant during this project. It helped me with writing and debugging code, planning the experiments, and drafting and formatting the report. I ran all code myself, checked the outputs and videos, and made the final decisions about which methods to keep. I can explain every method in this report.